Clase D_semana01:

Esta clase es el punto de entrada del programa. Su función principal es mostrar un menú interactivo al usuario y gestionar las opciones que este elija.

```java
public class D_semana01 {
    public static void main(String[] args) throws SQLException, ClassNotFoundException {
        Scanner sc = new Scanner(System.in);
        LogCliente objLogCliente = new LogCliente();
        int opc;

        do {
            System.out.println("\n===== MENU =====");
            System.out.println("1. Insertar cliente");
            System.out.println("2. Buscar cliente");
            System.out.println("3. Actualizar cliente");
            System.out.println("4. Listar clientes");
            System.out.println("0. Salir");
            System.out.print("Elige una opcion: ");
            opc = sc.nextInt();
            sc.nextLine();
```

Se utilizan dos objetos principales: un Scanner para leer los datos ingresados por consola, y un objeto LogCliente que conecta la presentación con la capa de lógica

El menú se muestra dentro de un bucle do-while, lo que garantiza que siempre se presente al menos una vez. El bucle se repite hasta que el usuario ingrese la opción 0 para salir.

Cada opción del menú corresponde a una operación sobre los clientes:

```java
switch (opc) {
    case 1:
        System.out.print("Cedula: ");
        String cedula = sc.nextLine();
        System.out.print("Nombre: ");
        String nombre = sc.nextLine();
        Cliente objCliente = new Cliente(cedula, nombre);
        if (objLogCliente.ValidarCedula(objCliente))
            System.out.println("Cliente insertado!");
        else
            System.out.println("Cedula incorrecta");
        break;

    case 2:
        System.out.print("Cedula a buscar: ");
        String cedulaBuscar = sc.nextLine();
        Cliente encontrado = objLogCliente.BuscarCliente(new Cliente(cedulaBuscar, nombre: ""));
        if (encontrado != null) {
            System.out.println("Cedula: " + encontrado.getCedula());
            System.out.println("Nombre: " + encontrado.getNombre());
        } else {
            System.out.println("Cliente no encontrado");
        }
        break;
```

- **Opción 1 – Insertar:** Solicita la cédula y nombre, crea un objeto Cliente y lo envía a la lógica para validar e insertar en la base de datos.

- **Opción 2 – Buscar:** Solicita una cédula, consulta la base de datos y muestra los datos del cliente si existe.

```java
            case 3:
                System.out.print("Cedula actual: ");
                String cedulaVieja = sc.nextLine();
                System.out.print("Nueva cedula: ");
                String cedulaNueva = sc.nextLine();
                System.out.print("Nuevo nombre: ");
                String nombreNuevo = sc.nextLine();
                boolean actualizado = objLogCliente.ActualizarCliente(cedulaVieja, new Cliente(cedulaNueva, nombreNuevo));
                if (actualizado)
                    System.out.println("Cliente actualizado!");
                else
                    System.out.println("Cedula no encontrada");
                break;

            case 4:
                ArrayList<Cliente> lista = objLogCliente.ListarClientes();
                for (Cliente c : lista) {
                    System.out.println(c.getCedula() + " - " + c.getNombre());
                }
                break;

            case 0:
                System.out.println("Hasta luego!");
                break;

            default:
                System.out.println("Opcion invalida");
        }

    } while (opc != 0);
    sc.close();
  }
}
```

- **Opción 3 – Actualizar:** Solicita la cédula actual del cliente, luego los nuevos datos, y actualiza el registro en la base de datos.
- **Opción 4 – Listar:** Obtiene todos los clientes registrados y los muestra por consola uno a uno.
- **Opción 0 – Salir:** Termina el bucle y cierra el Scanner.

Clase LogCliente

Esta clase actúa como intermediaria entre la presentación y la base de datos. Su responsabilidad es procesar los datos, aplicar validaciones y retornar resultados. No muestra nada por pantalla, solo trabaja con los datos. Contiene un objeto BDCliente que le permite comunicarse con la base de datos

- **Validar Cedula:**

```java
package Logica;
import BD.BDCliente;
import Clases.Cliente;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;

public class LogCliente {  3 usages
    BDCliente objBDCliente = new BDCliente();  4 usages
    public boolean ValidarCedula(Cliente objCliente)  1 usage
            throws ClassNotFoundException, SQLException {
        if (objCliente.getCedula().length()==10)
            objBDCliente.InsertarCliente(objCliente);
        else
            return false;
        return true;
    }
```

Recibe un objeto **Cliente** y verifica que su cédula tenga exactamente 10 caracteres antes de insertarlo. Si cumple la condición, llama a la base de datos para insertar el cliente y retorna *true*. Si la cédula no tiene 10 caracteres, retorna *false* sin realizar ninguna operación.

- **BuscarCliente**

```java
public Cliente BuscarCliente(Cliente cliente)  1 usage
        throws SQLException, ClassNotFoundException {
    ResultSet rs = objBDCliente.BuscarClienteCedula(cliente);
    if(rs.next()){
        return new Cliente(rs.getString( columnLabel: "cedula"), rs.getString( columnLabel: "nombre"));
    }
    return null;
}
```

Recibe un objeto **Cliente** que contiene únicamente la cédula a buscar. Envía ese dato a la base de datos y recibe un *ResultSet* con el resultado. Si encuentra el registro, crea un nuevo objeto Cliente con los datos reales obtenidos de la base de datos y lo retorna. Si no existe ningún cliente con esa cédula, retorna *null*.

- **ActualizarCliente**

```
public boolean ActualizarCliente(String cedulaVieja, Cliente cliente)  1 usage
        throws SQLException, ClassNotFoundException {
    return objBDCliente.ActualizarClienteCedula(cedulaVieja, cliente);
}
```

Recibe dos parámetros: la cédula actual para ubicar el registro, y un objeto **Cliente** con la nueva cédula y nombre. Delega la operación directamente a la base de datos y retorna *true* si la actualización fue exitosa, o *false* si no encontró ningún registro con esa cédula.

- **ListarClientes**

```
public ArrayList<Cliente> ListarClientes()  1 usage
        throws SQLException, ClassNotFoundException {
    ResultSet rs = objBDCliente.ListarClientes();
    ArrayList<Cliente> lista = new ArrayList<>();
    while (rs.next()) {
        lista.add(new Cliente(rs.getString( columnLabel: "cedula"), rs.getString( columnLabel: "nombre")));
    }
    return lista;
}
```

No recibe ningún parámetro. Consulta todos los clientes registrados en la base de datos, los recorre uno a uno con un *while* y por cada registro crea un objeto **Cliente** que agrega a un *ArrayList*. Al finalizar retorna la lista completa a la presentación.

Clase BDCliente

Esta clase es la única que se comunica directamente con MySQL. Su responsabilidad es ejecutar las sentencias SQL y retornar los resultados a la capa de lógica. Contiene un objeto Conexion que le proporciona la conexión a la base de datos.

```
package BD;
import Clases.Cliente;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
/**
 * @author LucasJG
 * Clase para CRUD de los clientes
 */

public class BDCliente {  3 usages
    Conexion BLcon = new Conexion();  4 usages
```

- **InsertarCliente**

```
public int InsertarCliente(Cliente objCliente)  1 usage
        throws ClassNotFoundException, SQLException {
    String Sentencia = "insert into clientes (cedula, nombre, idclient)"
            + "values (?,?,?)" ;
    PreparedStatement ps = BLcon.getConnection().prepareStatement(Sentencia);
    ps.setString( parameterIndex: 1,objCliente.getCedula());
    ps.setString( parameterIndex: 2,objCliente.getNombre());
    ps.setInt( parameterIndex: 3,  x: 0);
    return ps.executeUpdate();
}
```

Recibe un objeto **Cliente** y ejecuta una sentencia **INSERT** para registrarlo en la tabla clientes. Usa *PreparedStatement* para asignar los valores de cédula, nombre e idClient de forma segura. Retorna un entero indicando cuántas filas fueron afectadas.

- **BuscarClienteCedula**

```
public ResultSet BuscarClienteCedula(Cliente objCliente)  1 usage
        throws SQLException, ClassNotFoundException {
    String Sentencia = "SELECT * FROM Clientes where cedula = ?";
    PreparedStatement ps= BLcon.getConnection().prepareStatement(Sentencia);
    ps.setString( parameterIndex: 1, objCliente.getCedula());
    return ps.executeQuery();
}
```

Recibe un objeto **Cliente** con la cédula a buscar y ejecuta un **SELECT** filtrando por esa cédula. Retorna un *ResultSet* con la fila encontrada, que luego será procesada por la capa de lógica.

- **ActualizarClienteCedula**

```
public boolean ActualizarClienteCedula(String cedulaVieja, Cliente objCliente)  1 usage
        throws ClassNotFoundException, SQLException {
    String Sentencia = "UPDATE Clientes SET cedula = ?, nombre = ? WHERE cedula = ?";
    PreparedStatement ps = BLcon.getConnection().prepareStatement(Sentencia);
    ps.setString( parameterIndex: 1, objCliente.getCedula());
    ps.setString( parameterIndex: 2, objCliente.getNombre());
    ps.setString( parameterIndex: 3, cedulaVieja);
    return ps.executeUpdate() > 0;
}
```

Recibe la cédula actual como texto y un objeto Cliente con los nuevos datos. Ejecuta un UPDATE que modifica la cédula y el nombre del registro que coincida con la cédula actual en el WHERE. Retorna true si al menos una fila fue modificada, o false si no encontró el registro.

- **ListarClientes**

```
public ResultSet ListarClientes()  1 usage
        throws ClassNotFoundException, SQLException {
    String Sentencia = "SELECT * FROM Clientes";
    PreparedStatement ps = BLcon.getConnection().prepareStatement(Sentencia);
    return ps.executeQuery();
}
```

No recibe parámetros. Ejecuta un SELECT * para obtener todos los registros de la tabla clientes y retorna el ResultSet completo para que la lógica lo procese.